

Session 3 · Deck 3.B1

Your Toolkit: Python, Environments, and Version Control

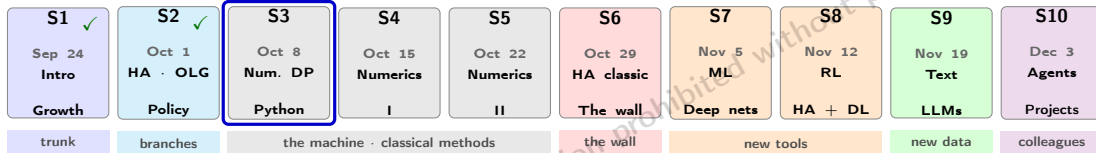
Zhigang Feng

Thursday, October 8, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 3.B1 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 3.B1 your toolkit · 3.B2 from loops to arrays · 3.B3 PyTorch, and coding with AI · 3.B4 the data workflow · 3.B5 hands-on: build it yourself.

Hands-on, 3.B5:

1 Version 0: grid VFI
against the closed form

2 Data → parameters:
 $g, n, \alpha, \delta, \beta$

3 The calibrated economy,
solved and checked

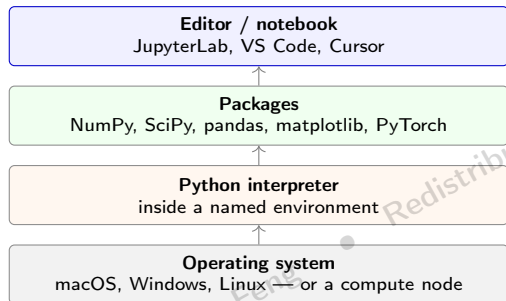
4 Test the model: untargeted
ratio, Euler residual

Four Error Messages You Are Going to Meet

What you will see	What it means	Fixed by
<code>ModuleNotFoundError: no module named 'numpy'</code>	the package is not installed <i>in the interpreter you are using</i>	environments
<code>command not found: python</code>	the shell cannot find an interpreter at all	installation, PATH
<code>AttributeError</code> on a function that exists in the docs	you are running a different version than the docs describe	environments
"It worked yesterday."	you changed something and cannot say what	version control

Not one of these is a Python problem. They are all **environment** problems, and they will cost you more hours this quarter than any algorithm in Block A. This deck is what prevents them.

How the Pieces Fit Together



The distinction that matters

The editor is *not* the thing that runs your code. It sends text to an interpreter, and which interpreter it picks is a setting — usually the one you forgot to check.

Every “it works for you but not for me” in this course is a disagreement about one of the middle two layers.

One Step Deeper: Interpreter, Operating System, Hardware

Follow one line of `vfi.py`, `c = y - k`, down to the machine that computes it:

Layer	What it is	What it does with <code>c = y - k</code>
Interpreter	python: a program written in C, compiled for your machine; each environment has its own	turns the line into bytecode — <code>LOAD_NAME y</code> , <code>LOAD_NAME k</code> , <code>BINARY_OP (-)</code> , <code>STORE_NAME c</code> — and runs it step by step, checking types and building a new float
Library	NumPy: machine code built ahead of time from C and Fortran	on arrays, one call: the loop over the elements runs as machine code
Operating system	the one program allowed to touch the hardware	starts python as a process (found on <code>PATH</code>), gives it memory and a core, reads <code>vfi.py</code>
Hardware	a CPU, whose machine code is x86-64 or ARM64 (Apple silicon)	the subtraction itself: one instruction (<code>subsd</code> on x86-64), in the float64 of 3.A1

Only the bottom layer computes; the rest translates. A Python loop pays the translation once per *element*, NumPy once per *array*: the gap 3.B2 measures.

Across machines: compiled code runs only on the OS and CPU it was built for, so rebuild an environment from `requirements.txt`; never copy it.

Where You Will Actually Type

Tool	Best for
JupyterLab	exploring, plotting, homework you hand in
VS Code / Cursor	multi-file projects, debugging, AI assistance inline
PyCharm	large codebases, heavy refactoring

For this course: notebooks for the labs and homework, an editor once your project outgrows one file — which will happen in Session 5.

A notebook is not a program

Cells can be run out of order, so a notebook that looks correct may depend on a variable you have since deleted.

Before you submit anything: **Restart Kernel and Run All**. If it fails, it was never working.

The same discipline is why the reference numbers in Block A were produced by a script submitted to a compute node, not by a notebook someone had been clicking around in.

Language, Library, Model — Three Different Things

- **Python** is the language: syntax, control flow, functions, classes. It is slow, and that is fine, because it is glue.
- **NumPy, SciPy, PyTorch** are libraries: compiled code with a Python interface. The arithmetic happens in C or CUDA, not in your loop.
- **An AI assistant** is a model behind an interface. It writes Python; it does not run it, and it does not know whether the answer is right.

Why this matters for speed

Python being slow is only a problem when *Python* does the arithmetic. Push the loop into the library and the language stops mattering — which is the whole of 3.B2.

And why it matters for trust: the library is tested by thousands of users; the assistant's output has been tested by nobody.

Installing: Who Does What

Anaconda / Miniconda bundles three separate things people confuse: a Python interpreter, a package manager, and an environment manager. Miniconda is the same thing without the 3 GB of packages you will not use.

An editor is not a distribution. Installing VS Code does not install Python; it goes looking for one. On

Windows, tick “Add to PATH”. Almost every command not found traces back to that box.

```
$ python --version
```

```
Python 3.11.9
```

```
$ which python
```

```
/opt/miniconda3/envs/econ282e/bin/python
```

Run both of these *before* asking anyone for help. They answer “which Python am I actually using?”, which is the first question every answer depends on.

Environments: One Habit, Most of the Breakage Gone

An environment is a private folder holding one interpreter and one set of package versions. Projects stop fighting over versions because they never share any.

```
conda create -n econ282e python=3.11
conda activate econ282e
pip install numpy scipy pandas \
    matplotlib jupyter torch tqdm \
    statsmodels pandas-datareader
pip freeze > requirements.txt
```

Two rules

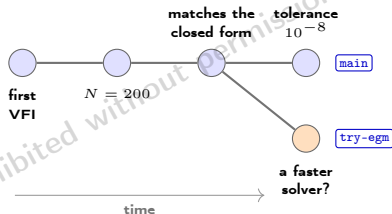
1. Packages are installed *into* an environment. Installing while the wrong one is active is the single most common cause of `ModuleNotFoundError`.
2. Prefer conda *or* pip within one environment; mixing them for the same package leads to states neither tool can repair.

`requirements.txt` is what makes your results reproducible by someone else — including the version of you who returns in March. **Lab L00** walks you through this setup and checks it.

Git: What It Is

Git is a **version-control system**: it keeps the history of a project folder, so that “it worked yesterday” becomes a question you can answer.

- A **commit** is a snapshot of every tracked file, with an author, a time, a message, and a link to the commit before it.
- The **history** is the chain of commits. Any of them can be inspected, compared with today, or brought back.
- A **branch** is a movable name for one line of commits: `main`, or an experiment you may throw away.
- The folder plus its history is a **repository**. Every copy of it carries the *whole* history.



Git is not GitHub

Git is a program on your machine; it works offline. **GitHub** is a website that keeps a copy of your repository — a *remote* — and adds tools for sharing and review. GitLab and Bitbucket do the same job.

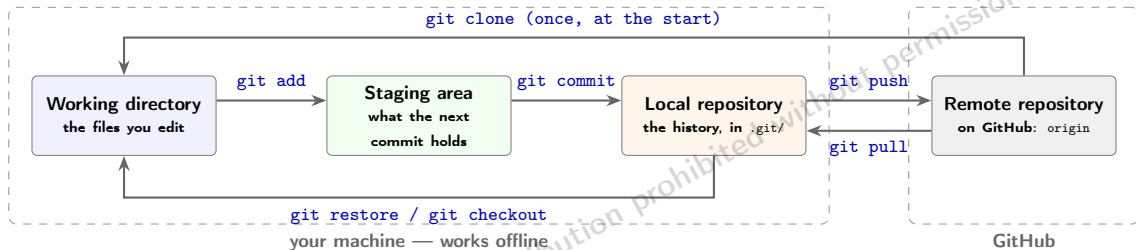
Reference: Chacon & Straub, *Pro Git*, 2nd ed. (2014).

What Git Can Do for You

You want to	Git does it with	Where you meet it here
see what changed since the last snapshot	<code>git status</code> , <code>git diff</code>	before every commit
record a result	<code>git add</code> , <code>git commit</code>	each time a number you report changes
find when it broke, and go back	<code>git log</code> , <code>git diff <commit></code> , <code>git restore</code> , <code>git checkout <commit></code>	"it worked yesterday"
try an idea without risking what works	<code>git switch -c <branch></code> , <code>git merge</code>	a faster solver beside the one you trust
work with other people	<code>git push</code> , <code>git pull</code> ; a <i>pull request</i> on GitHub	the final project, in teams of 2–3
hand in something that reproduces	a repository with <code>run_all.py</code>	HW1, HW2, the project's replication package

One idea is underneath all six rows: every commit is kept. **Nothing you committed is ever lost** — only what you never committed. Homework is a PDF, the code that produced it, and an `AI_LOG.md`; Git is how you make sure the three agree.

The Structure: Your Machine and GitHub



Three places on your machine, one on GitHub. You edit in the working directory; add chooses which changes go into the next snapshot; commit writes it into the history. Your history and GitHub's meet only when you push or pull.

A commit is **local**. Until you `git push`, your history lives on one disk only — and your teammates cannot see it.

The three local areas follow Chacon & Straub, *Pro Git*, 2nd ed. (2014), ch. 1, "What is Git?".

One File, Three Edits: Before and After You Commit

`vfi.py` is committed with `beta = 0.95`, `N = 200`, `tol = 1e-6`. You edit it three times: `N` to 400, `tol` to `1e-8`, and — a typo — `beta` to 9.5.

Save three times, commit later

```
$ git status --short
M vfi.py
$ git log --oneline
b434f81 first VFI: 272 sweeps, err 0.99%
$ git diff
-beta = 0.95
-N = 200
-tol = 1e-6
+beta = 9.5
+N = 400
+tol = 1e-8
```

Git does not see saves: the three edits are **one tangled change**, and `git restore vfi.py` removes the typo only by discarding the good edits too.

Commit after each save

```
$ git log --oneline
b07c7af update beta
0aab4c6 tol 1e-8
2778c70 grid N=400
b434f81 first VFI: 272 sweeps, err 0.99%
$ git diff HEAD~1 HEAD
-beta = 0.95
+beta = 9.5
N = 400
tol = 1e-8
$ git revert HEAD # undo that commit only
```

Each commit is a snapshot you can name, compare and return to. The typo sits **alone** in one commit: `git revert` removes it and keeps the other two.

A save changes the file; only a commit changes the history.

The Daily Loop

```
git clone https://github.com/you/hw1
cd hw1                                # once, at the start

git pull                             # your team's work
# ... edit vfi.py, run it ...
git status                           # what changed?
git diff                             # how, line by line?
git add vfi.py
git commit -m "VFI: 272 sweeps, err 0.99%"
git push                             # history -> GitHub
git log --oneline                    # the story so far
```

GitHub stopped accepting account passwords for `git push` in 2021: set up an SSH key, or sign in once through GitHub Desktop or the `gh` command-line tool.

Commit when a number changes

The useful habit is not daily commits — it is committing at the moment a result becomes reproducible, with the result in the message.

Never commit data you do not own, model checkpoints, or anything with a password in it. A `.gitignore` is written once per project and saves you from all three.

Keep the Repository Out of OneDrive and Dropbox

A sync service — OneDrive, Dropbox, iCloud Drive, Google Drive — also keeps a folder's history, but by a different rule:

	Cloud sync	Git
What it saves	each file, on its own	the whole project at once
When	whenever a file changes	when you commit
Message	none	one per commit
Two machines edit one file	keeps both, as a renamed "conflicted copy"	git pull merges, or shows the conflict line by line

Use one system per folder. For code, that is Git, and GitHub is the backup.

Why the two collide

.git/ is many small files that must change **together**. The sync uploads them one at a time, so another machine can get half a commit: bad object, index file corrupt. A synced lock file blocks the next commit (index.lock: File exists), and online-only placeholders leave files Git cannot read.

The rule

Keep repositories in a plain local folder (C:\code, ~/code), and **check it really is outside**: Windows often moves Documents and Desktop into OneDrive, and macOS can put them in iCloud Drive.

When You Need a Bigger Machine

Nothing in Block A needed more than a laptop: the entire morning's computation ran in under a second. That stops being true in Sessions 7 and 8, when training a network replaces iterating an operator.

- **Google Colab** gives a free GPU in a browser. Good for a lab, bad for anything you need to reproduce — the machine is gone when you close the tab.
- **A cluster** gives you a scheduler. You write a job script, submit it, and collect the output; the login node is for editing, never for computing.

The portable habit

Put the computation in a **script** that takes its settings from the top of the file and writes its results to disk. Then the same file runs on a laptop, in Colab, or under a scheduler, unchanged.

That is exactly how this morning's numbers were produced: one script, one output file, and every figure drawn from it.

The Four Atomic Types

```
year      = 2026          # int
inflation = 0.031         # float
country   = "Chile"       # str
recession = False         # bool

type(inflation)           # <class 'float'>
int("2026") + 1           # 2027
float("3.1") * 2          # 6.2
```

Python infers the type; you do not declare it. That is convenient and occasionally dangerous.

1/2 is 0.5 in Python 3 but 0 in Python 2 and in most of C. Integer division is written `//` — and is exactly how you index into a grid.

A float here is the float64 of 3.141592653589793: about sixteen significant digits, and gaps that widen as the numbers grow.

Lists, Slicing, and Text

```
g = [2.1, 2.9, -0.4, 1.8, 2.5]

g[0]          # 2.1    first
g[-1]         # 2.5    last
g[1:3]        # [2.9, -0.4]
g[:2], g[3:]
len(g), sum(g) / len(g)

g.append(3.0)      # modifies g
sorted(g)[:2]      # two worst years

name = "  gdp Growth "
name.strip().title() # 'Gdp Growth'
f"inflation was {0.031:.1%}"
```

Slices are half-open: `g[1:3]` gives elements 1 and 2. It reads badly and it is the right convention — `g[:i] + g[i:]` reconstructs the list for every `i`.

Indexing from **zero** and slicing half-open are the two conventions behind every off-by-one error you will write this quarter. They are also what make `k[i]`, `k[i+1]` in an interpolation formula line up.

The One That Bites: Reference or Copy?

```
a = [1, 2, 3]
b = a          # NOT a copy
b[0] = 99
a              # [99, 2, 3]

c = a.copy()   # a real copy
c[0] = 1
a              # [99, 2, 3]
```

`b = a` makes a second *name* for one object.

Assignment never copies.

Why this will bite you

The same rule governs NumPy arrays and PyTorch tensors — and slices of them are **views**, not copies. In a value-function loop, `V_new = V` followed by an update silently destroys the very thing you wanted to compare against.

That is why the loop in 3.B2 and Lab L03 writes `V, n = Vn, n + 1` — binding a *new* array — and never writes into the old one.

Dictionaries, Tuples, Sets

```
par = {"alpha": 0.36, "beta": 0.95,
      "delta": 1.0}
par["beta"]           # 0.95
par["sigma"] = 0.007   # add a key
for k, v in par.items():
    print(k, v)

bounds = (0.05, 0.60)  # tuple: fixed
# bounds[0] = 0.1      -> TypeError

set([1, 1, 2, 3])      # {1, 2, 3}
```

- **Dictionaries** are how you carry a calibration around. One object, named fields, and it serialises straight to JSON — which is how a results file records the parameters that produced it.
- **Tuples** are immutable. Use them for things that must not change: grid bounds, an array shape, a pair of coordinates.
- **Sets** deduplicate and test membership fast.

Takeaways

- The errors that cost you time this quarter will be **environment** errors, not Python errors. One environment per project, created on the first day, removes most of them.
- `which python` and `python --version` answer the question every other answer depends on. Run them before asking for help.
- The editor does not run your code and a notebook is not a program. **Restart and Run All** before you submit anything.
- Git keeps your project's history on your machine; GitHub keeps a copy others can reach. `add`, `commit`, `push`: commit when a number changes, and put the number in the message. Keep the repository out of OneDrive and Dropbox.
- Assignment never copies. `b = a` gives one object two names — and the same rule governs NumPy arrays and PyTorch tensors, where slices are views.

Bridge: The Algorithm Is on the Board. Put It in an Array.

You now have a machine that runs Python and a project that can be reproduced. What you do not have is the one idea that separates code which finishes from code which does not.

This morning's Bellman update is a double loop: for every state, for every choice. Written that way in Python it is unusably slow. Written as a single array expression it is the same arithmetic, the same answer to the last bit — and it runs a few **hundred times faster**.

Next (3.B2)

Control flow, functions and classes — then NumPy, broadcasting, and the vectorised Bellman update.

We rebuild this morning's solver as a `GrowthModel` class, and measure what each change bought.